

SUBPROGRAMS PART 2

-functions, packages, triggers-

Subprograms packages

The packages group together variables, subprograms, PL/SQL data types, all of them being logically correlated.

They have 2 parts that are compiled separately:

- **Package specification** – public
- **Package body** – private

Package specification

```
CREATE [OR REPLACE] PACKAGE package_name
IS|AS
-- declaring public variables and types, implicitly initialized with NULL
-- specifications for public subprograms
END [package_name];
```

Package body

```
CREATE [OR REPLACE] PACKAGE BODY package_name
IS|AS
-- declaring private variables and types
-- defining public and private subprograms
[BEGIN
-- it is optional and it's executed only once at the package's first call
END [package_name];
```

Remark: In a package, in order to use a function/procedure in another subprogram, it has to be previously declared (forward declarations principle);

Overloading the subprograms:

- It is possible only for functions/procedures that are contained in packages, not stored on their own in the database.
- Subprograms with similar type parameters cannot be overloaded (ex: NUMBER and DECIMAL or VARCHAR2 and VARCHAR);

Example: Create a package that contains a private procedure and a public function. The procedure returns the number of customers that made no orders and the function displays that number and returns YES or NO if a customer made orders or not. Create a body for the package and handle a user defined exception in the function, if the number that should be returned is higher than 100.

Create or replace package new_package
is

```
FUNCTION no_cust (customer in customers.customer_id%type) return varchar2;
end;
```

```
Create or replace package body new_package
is
PROCEDURE count_customers (no out number)
is
cursor c is select customer_id from customers;
i number(4);
begin
no:=0;
for rec in c loop
select count(order_id) into i from orders where customer_id=rec.customer_id;
if i=0 then no:=no+1;
end if;
end loop;
end count_customers;

FUNCTION no_cust (customer in customers.customer_id%type) return varchar2
is res number(4);
i number(4);
exc exception ;
begin
count_customers(res);
dbms_output.put_line('There are '||res||' customers with no orders');
select count(order_id) into i from orders where customer_id=customer;
if i=0 then return 'NO';
elsif i>100 then raise exc;
else return 'YES';
END IF;
EXCEPTION WHEN exc then return 'exception';
end no_cust;
begin
dbms_output.put_line('Message inside a package body');
end;
```

The call of the subprograms contained in the package:

```
declare
var varchar2(10);
begin
var:=new_package.no_cust(102); --216
dbms_output.put_line(var);
end;
```

Call the function in a SELECT using DUAL:

```
select new_package.no_cust(102) from dual
```

Call the function in a SQL statement:

select customer_id, new_package.no_cust(customer_id) as "did the customer make orders" from customers;

Remarks!

The functions used in SQL expressions must have only IN parameters and must return only SQL data. They cannot include DML, DDL or transaction control commands (commit, rollback) and they must not call other subprograms that do that.

Deleting the package:

DROP PACKAGE package_name;

DROP PACKAGE BODY package_name;

Triggers

TRIGGER = a type of subprogram that is executed automatically when a certain event or action happens.

When declaring the trigger the following components must be specified:

- **the trigger's type** (after, before, instead of),
- **the triggering event** (pressing a key, mouse click, a certain statement, etc.),
- **the action** that takes place when the event happens (a set of statements).

Calling the trigger is made automatically whenever the event triggers.

- The syntax for creating a trigger is:

```
CREATE [OR REPLACE] TRIGGER trigger_name
[BEFORE| AFTER| INSTEAD OF –only for views]
[INSERT| [OR] | UPDATE [OF column,...]| [OR] | DELETE]
ON table
[FOR EACH ROW ]
[WHEN condition]
-- Trigger body
```

The **trigger's body** can be a PL/SQL block or a procedure's call. The procedure can be implemented in PL/SQL, C or Java and the call is made like this: *CALL procedure_name* (without ; after its name!!!)

Example: Create a trigger that is raised before each update operation on Product_information table.

```
CREATE OR REPLACE TRIGGER products_trig
BEFORE update ON product_information
BEGIN
dbms_output.put_line('the trigger executed');
END;
```

```
update product_information set min_price=100 where product_id<1730;
```

INSERTING, UPDATING, DELETING

Example: Create a trigger that reacts differently for each operation on Product_information table.

```
CREATE OR REPLACE TRIGGER products_log
BEFORE INSERT or UPDATE or DELETE on product_information
DECLARE
v_type varchar2(1);
BEGIN
  case
    when INSERTING then v_type := 'I';
    when UPDATING then v_type := 'U';
    ELSE v_type := 'D';
  END case;
  dbms_output.put_line(user||sysdate||' '||v_type);
END;
```

FOR EACH ROW triggers

These type of triggers can access the current row using 2 **pseudo-records** :old, :new. Their type is:

Table_on_which_trigger_acts%ROWTYPE

The values for :old and :new

Operation	Value for Old	Value for New
INSERT	NULL	New value, inserted
UPDATE	Old value, previous the <i>update</i>	New value, updated
DELETE	Old value, previous the <i>delete</i>	NULL

- (:OLD) is not defined for INSERT and (:NEW) is not defined for DELETE
- Although, syntactically they are treated like record, they are not records, so the operations for assignment var_record:=:old are not valid for them. This is why the fields have to be specified for the pseudo-records: :old.field or :new.field.
- For each row triggers that use columns of tables that are involved in DML commands that cause the trigger to react are not recommended. A *mutating table* error will raise.
- WHEN clause can be used to put a condition on a for each row trigger. The trigger's body will be executed only for those rows that meet the condition.

Example: Create a trigger to restrict the maximum price allowed for a product.

```
CREATE OR REPLACE TRIGGER restrict_price
BEFORE INSERT or UPDATE on product_information
FOR EACH ROW
WHEN (new.product_status='orderable')
BEGIN
  IF :new.list_price<:new.min_price then
    RAISE_APPLICATION_ERROR (-20202, 'The price has to be higher than the min price!');
  end if;
END;
```

```
/
update product_information
set list_price = min_price-1
where product_id=2384; -- 2414
```

Managing triggers:

- A trigger can be disabled like this:

```
ALTER TRIGGER trigger_name DISABLE|ENABLE;
```

or:

```
ALTER TABLE table_name
DISABLE|ENABLE ALL TRIGGERS;
```

- Recompiling a trigger:

```
ALTER TRIGGER trigger_name COMPILE;
```

- A trigger can be deleted:

```
DROP TRIGGER trigger_name;
```

When a table is deleted, all its triggers are automatically deleted.

Homework

- 1) Create an *after logon on database* trigger
Create table logins (the_date date);

```
create or replace trigger
logon_trigg after logon on database
begin
insert into logins values (sysdate);
end;
```

- 2) Create a trigger that maintains the unique constraint for the product_id in product_information table, using values generated by a sequence.
create sequence seq start with 4000 increment by 1 nocycle;

```
create or replace trigger insert_trigg
before insert on product_information
for each row
begin
select seq.nextval into :new.product_id from dual;
end;
```

```
insert into product_information(product_name) values ('something');
```